03 — Estados

Versão 1.2 · atualizado em 08/11/2026

Este documento define as máquinas de estado de **Missão**, **Agente** e **Anomalia** do Continuum Control: quais transições existem, o que precisa ser verdade para cada uma acontecer (guarda) e o que ela provoca (ação). As referências `§N` apontam para o enunciado; `D` e `A` continuam a numeração dos docs 01 e 02.

Diagramas	Blocos Mermaid neste documento, renderizados pelo próprio GitHub, e páginas *Estados — Missão*, *Estados — Agente* e *Estados — Anomalia* em `diagramas/continuum-control.drawio`, para edição e apresentação
Verificação	`verificacao/modelo-estados.py`: modelo executável das três máquinas (seção 8) · `verificacao/confere-diagramas.py`: confere draw.io × Mermaid × tabelas (seção 8.3)
Documentos relacionados	00 — Enunciado · 01 — Modelo de domínio · 02 — Modelo de dados · 04 — Persistência · 05 — API · 06 — Segurança

Sumário

1. Visão geral
2. Convenções
3. Missão
4. Agente
5. Anomalia
6. Operações compostas
7. Invariantes
8. Verificação
9. Decisões de modelagem
10. Pontos em aberto
11. Histórico de revisões

1. Visão geral

As três máquinas não são independentes: **a missão comanda as outras duas**. Iniciar e encerrar uma missão mudam, na mesma transação, o estado dos agentes e da anomalia (§9).

Operação	Missão	Agentes	Anomalia
Iniciar missão	→ `EM_EXECUCAO`	→ `EM_MISSAO`	→ `EM_ANALISE` ou `EM_CORRECAO`
Concluir missão	→ `CONCLUIDA`	→ `DISPONIVEL`	→ situação final do encerramento
Registrar falha	→ `FALHOU`	→ `DISPONIVEL` ou `SUSPENSO`	→ situação final do encerramento (nunca `ESTABILIZADA`)
Declarar anomalia irreversível	Correções pendentes → `CANCELADA`	—	→ `IRREVERSIVEL`

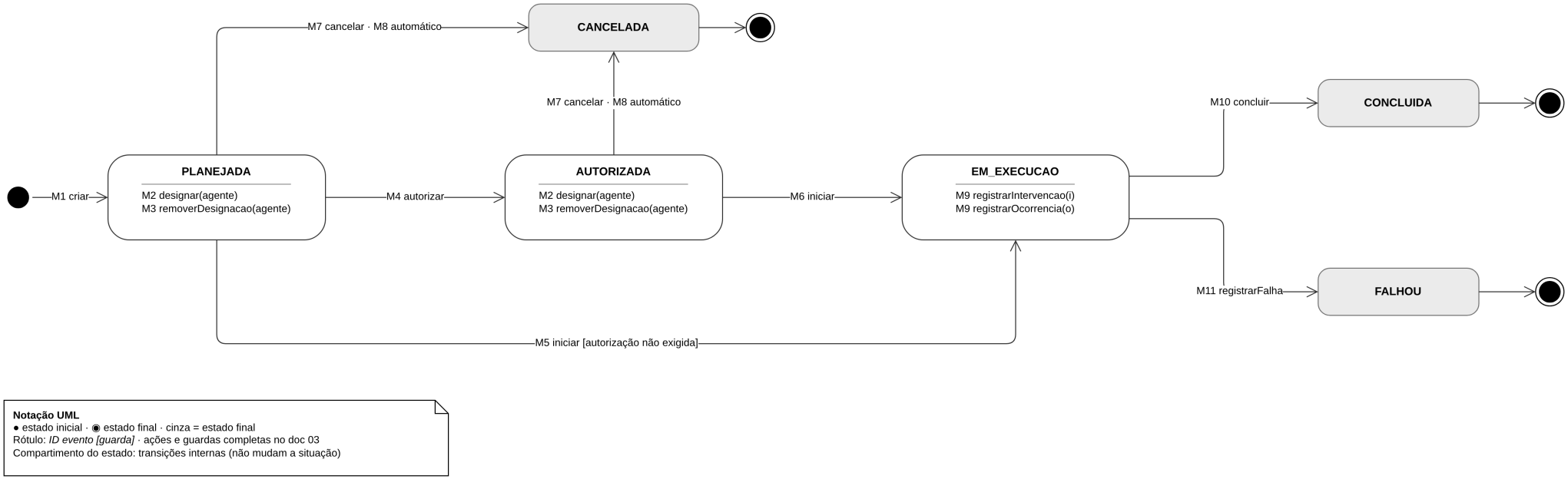
Fora dessas operações, cada máquina muda sozinha: o administrador suspende ou reativa um agente, e o risco de uma anomalia pode ser reclassificado a qualquer momento enquanto ela estiver ativa.

2. Convenções

Notação	Significado
`evento [guarda] / ação`	Notação UML. Nos diagramas aparecem só o ID e o evento; guardas e ações completas ficam nas tabelas
`M`, `G`, `N` + número	IDs das transições de Missão, aGente e aNomalia, usados no diagrama, nas tabelas e no modelo executável
● →	Estado inicial
→ ◉	Estado final: a entidade não muda mais de situação, mas continua consultável (§5, §12)
Transição interna	Operação que acontece **dentro** de um estado, sem mudar a situação (ex.: designar agente em `PLANEJADA`)
Automática	Transição disparada por uma operação em outra entidade, e não por uma requisição direta

Toda guarda violada resulta em `409 Conflict` (§18), com a regra violada na mensagem de erro.

3. Missão



Notação UML
● estado inicial · ◉ estado final · cinza = estado final
Rótulo: *ID evento [guarda]* · ações e guardas completas no doc 03
Compartimento do estado: transições internas (não mudam a situação)

página *Estados — Missão* de `diagramas/continuum-control.drawio` · equivalente ao bloco Mermaid do `.md`, conferido por `confere-diagramas.py` (seção 8.3)

3.1 Transições

ID	De → Para	Evento	Guarda	Ação
M1	● → `PLANEJADA`	`criar(anomalia, proposito, ...)`	`anomalia.podeReceberMissao(proposito)` (5.3)	Registro `MISSAO_CRIADA`
M2	interna (`PLANEJADA`, `AUTORIZADA`)	`designar(agente)`	Agente `DISPONIVEL` e ainda não designado nesta missão	Cria `MissaoAgente`; registro `AGENTE_DESIGNADO`
M3	interna (`PLANEJADA`, `AUTORIZADA`)	`removerDesignacao(agente)`	Agente designado nesta missão	Remove a linha de `missao_agente` (D25); registro `AGENTE_REMOVIDO`
M4	`PLANEJADA` → `AUTORIZADA`	`autorizar(responsavel, observacao)`	Responsável com perfil `SUPERVISOR` ou `ADMINISTRADOR` (§8, §22)	Cria `Autorizacao`; registro `AUTORIZACAO`
M5	`PLANEJADA` → `EM_EXECUCAO`	`iniciar()`	`¬exigeAutorizacao()` **e** `podeIniciar()`	Operação composta 6.1
M6	`AUTORIZADA` → `EM_EXECUCAO`	`iniciar()`	`podeIniciar()`	Operação composta 6.1
M7	`PLANEJADA` / `AUTORIZADA` → `CANCELADA`	`cancelar(motivo)`	—	Operação composta 6.3
M8	`PLANEJADA` / `AUTORIZADA` → `CANCELADA`	Automática: a anomalia deixou de admitir a missão (D24)	`¬anomalia.podeReceberMissao(proposito)`	Igual a M7, com o motivo gerado pelo sistema
M9	interna (`EM_EXECUCAO`)	`registrarIntervencao(i)` / `registrarOcorrencia(o)`	Autor designado na missão (§10, §11)	Grava a intervenção, ou a ocorrência pela operação 6.6; registro `INTERVENCAO` / `OCORRENCIA`
M10	`EM_EXECUCAO` → `CONCLUIDA`	`concluir(encerramento)`	Situação final permitida pela tabela 5.2	Operação composta 6.2
M11	`EM_EXECUCAO` → `FALHOU`	`registrarFalha(encerramento)`	Situação final permitida pela tabela 5.2	Operação composta 6.2

3.2 Guardas compostas

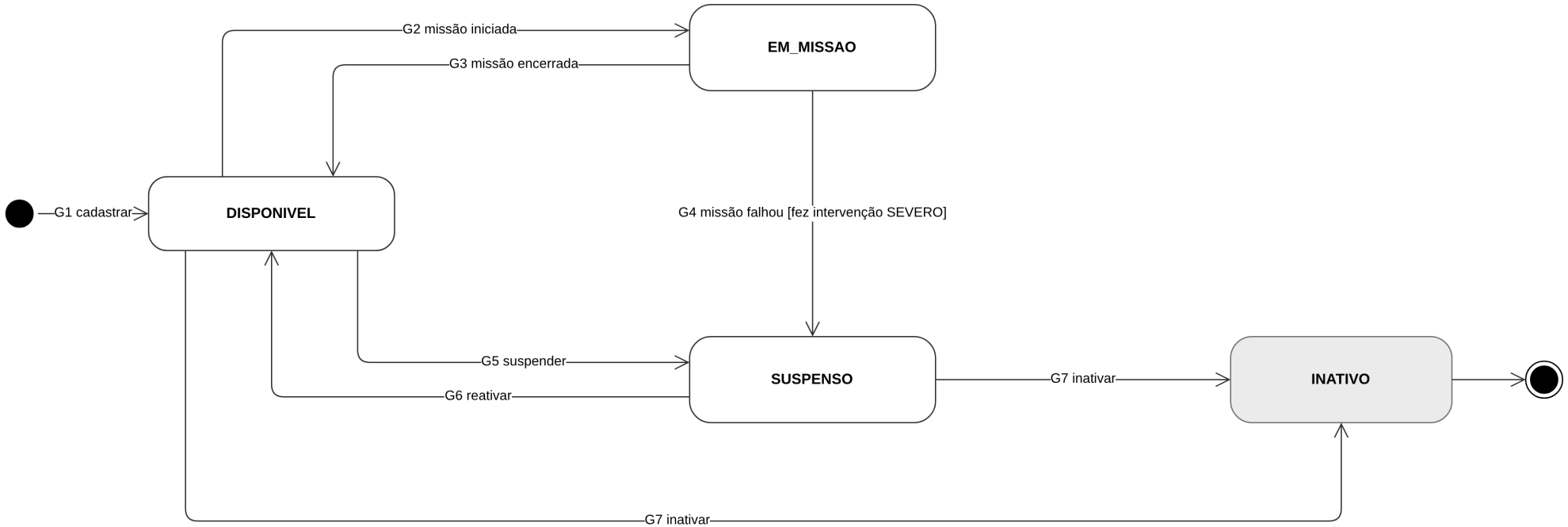
Guarda	Definição
`exigeAutorizacao()`	`prioridade = EMERGENCIA` **ou** `anomalia.risco = CRITICO` (§8, A5)
`podeIniciar()`	Todas as condições: ≥ 1 agente designado (§7); todos os designados `DISPONIVEL` (§3, §7, D11); `anomalia.podeReceberMissao(proposito)`; nenhuma outra missão `EM_EXECUCAO` para a mesma anomalia (D23)

`exigeAutorizacao()` é avaliada **no momento de iniciar**, e não na criação. Se o risco da anomalia subir para `CRITICO` depois que a missão foi planejada, ela passa a precisar de autorização antes de começar.

3.3 Outras regras

- **Edição de dados** (objetivo, destino temporal, prioridade): só em `PLANEJADA`. Depois de autorizada, a missão não muda o que foi autorizado (D28).
- **Equipe após a autorização:** M2 e M3 continuam permitidas em `AUTORIZADA`. A autorização vale para a missão, e as mudanças de equipe ficam no histórico (D28).
- **Remoção física:** nenhuma missão é removida (D3). Cancelar é a forma de desistir dela.

4. Agente



Notação UML
● estado inicial · ◉ estado final · cinza = estado final
Rótulo: *ID evento [guarda]* · ações e guardas completas no doc 03
Compartimento do estado: transições internas (não mudam a situação)

página *Estados — Agente* de `diagramas/continuum-control.drawio` · equivalente ao bloco Mermaid do `.md`, conferido por `confere-diagramas.py` (seção 8.3)

4.1 Transições

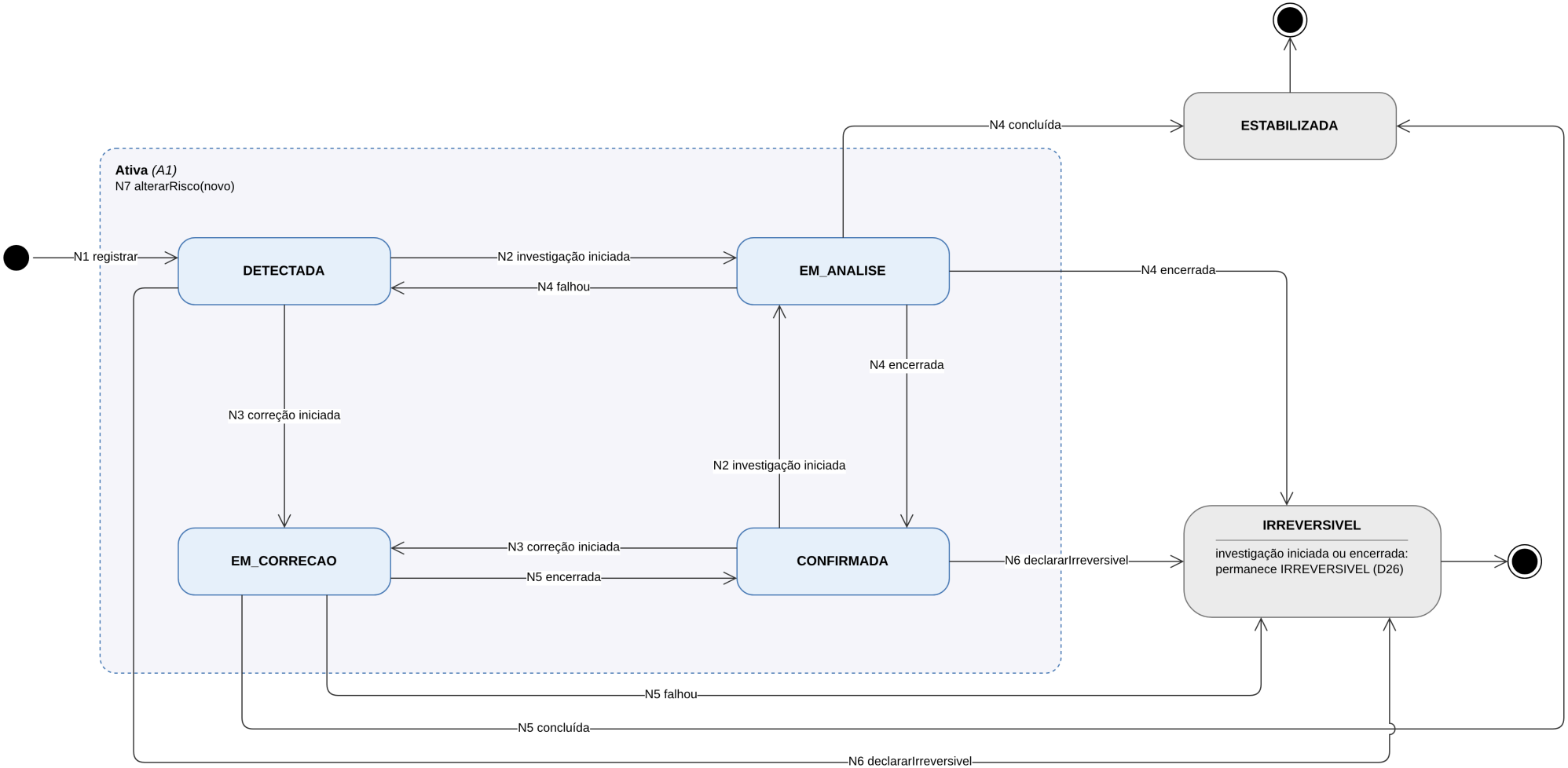
ID	De → Para	Evento	Guarda	Ação
G1	● → `DISPONIVEL`	`cadastrar(...)`	Código único (§3)	Registro de agente com situação anterior nula
G2	`DISPONIVEL` → `EM_MISSAO`	Automática: M5/M6	—	`entrarEmMissao()` ; registro de agente
G3	`EM_MISSAO` → `DISPONIVEL`	Automática: M10/M11	Não se aplica G4	`liberar()` ; registro de agente
G4	`EM_MISSAO` → `SUSPENSO`	Automática: M11	A missão **falhou** e o agente registrou nela uma intervenção com impacto `SEVERO` (A4, D30)	`suspenderPorFalha()` (doc 01, D34); registro de agente
G5	`DISPONIVEL` → `SUSPENSO`	`suspender(motivo)`	Perfil `ADMINISTRADOR`	Registro de agente com o motivo
G6	`SUSPENSO` → `DISPONIVEL`	`reativar(motivo)`	Perfil `ADMINISTRADOR`	Registro de agente com o motivo
G7	`DISPONIVEL` / `SUSPENSO` → `INATIVO`	`inativar(motivo)`	Perfil `ADMINISTRADOR`	Operação composta 6.5

"Registro de agente" é um `RegistroAgente` gravado na mesma transação: toda transição de G1 a G7 deixa um (doc 01, D31). Nas automáticas, o motivo é gerado pelo sistema e o usuário fica nulo.

4.2 Regras (fecham A2, A4 e A12)

- **Agente em missão não muda por ação manual.** Suspender ou inativar alguém `EM_MISSAO` exigiria deixar uma missão em execução sem o seu agente (§9). Primeiro a missão é encerrada; depois o agente muda.
- **Suspender um agente designado em missão pendente** é permitido. A missão não inicia enquanto ele estiver suspenso (guarda de `podeIniciar()`); o operador pode removê-lo (M3) ou esperar a reativação.
- **`INATIVO` é final** (D29). Um agente desligado não volta; ele continua nos registros (§3, §12).
- **Toda mudança de situação fica registrada**, com motivo, instante e autor, em `RegistroAgente` (A12, doc 01 D31).

5. Anomalia



Notação UML
● estado inicial · ◉ estado final · cinza = estado final
Rótulo: *ID evento [guarda]* · ações e guardas completas no doc 03
Compartimento do estado: transições internas (não mudam a situação)
N4/N5 *encerrada* = concluída ou falhou

página *Estados — Anomalia* de `diagramas/continuum-control.drawio` · equivalente ao bloco Mermaid do `.md`, conferido por `confere-diagramas.py` (seção 8.3)

Em azul, os estados **ativos**; em cinza, os finais. "Encerrada" no rótulo significa "concluída ou falhou".

5.1 Transições

ID	De → Para	Evento	Guarda	Ação
N1	● → `DETECTADA`	`registrar(evento, divergencia, risco, origem, observacoes, dataHoraDeteccao)`; os três últimos opcionais	Evento existente (§5); `dataHoraDeteccao` não está no futuro (padrão: agora)	—
N2	`DETECTADA` / `CONFIRMADA` → `EM_ANALISE`	Automática: início de missão de `INVESTIGACAO`	—	Registro `MUDANCA_SITUACAO`
N3	`DETECTADA` / `CONFIRMADA` → `EM_CORRECAO`	Automática: início de missão de `CORRECAO`	—	Registro `MUDANCA_SITUACAO`
N4	`EM_ANALISE` → situação final	Automática: encerramento de investigação	Tabela 5.2	Registro `MUDANCA_SITUACAO`; M8 se a nova situação for final (D24)
N5	`EM_CORRECAO` → situação final	Automática: encerramento de correção	Tabela 5.2	Igual a N4
N6	`DETECTADA` / `CONFIRMADA` → `IRREVERSIVEL`	`declararIrreversivel(justificativa)`	Perfil `SUPERVISOR` ou `ADMINISTRADOR`; nenhuma missão `EM_EXECUCAO` para a anomalia (conferido no serviço)	Operação composta 6.4
N7	interna (estados ativos)	`alterarRisco(novo)`	Anomalia ativa	Registro `MUDANCA_RISCO`

`EM_ANALISE` e `EM_CORRECAO` **só** são alcançados pelo início de uma missão, nunca por alteração manual (D27). Por isso a `Anomalia` não expõe um `alterarSituacao(nova)` genérico. Cada mudança tem um método com intenção (doc 01): `declararIrreversivel()` (N6) é público; `iniciarAtendimento(proposito)` (N2, N3) e `encerrarAtendimento(situacaoFinal)` (N4, N5) só são chamados pela `Missao`, dentro do pacote de domínio.

Investigação em anomalia irreversível: o §5 só bloqueia correção, então a investigação é permitida. Ela não muda a situação: a anomalia continua `IRREVERSIVEL` durante a missão e depois dela (D26).

5.2 Situação final no encerramento (fecha A8)

`Encerramento.situacaoFinalAnomalia` precisa estar nesta tabela:

Propósito	Desfecho	Situações finais permitidas
`INVESTIGACAO`	`CONCLUIDA`	`CONFIRMADA`, `ESTABILIZADA`, `IRREVERSIVEL`
`INVESTIGACAO`	`FALHOU`	`DETECTADA`, `CONFIRMADA`, `IRREVERSIVEL`
`CORRECAO`	`CONCLUIDA`	`ESTABILIZADA`, `CONFIRMADA` (correção parcial)
`CORRECAO`	`FALHOU`	`CONFIRMADA`, `IRREVERSIVEL`
`INVESTIGACAO` em anomalia `IRREVERSIVEL`	qualquer	`IRREVERSIVEL`

- Nenhuma linha com `FALHOU` admite `ESTABILIZADA`: uma falha não resolve a anomalia (§13).
- `EM_ANALISE` e `EM_CORRECAO` nunca são situação final; o banco também barra isso (`ck_encerramento_situacao`, doc 02).

5.3 Definições (fecha A1)

Termo	Definição	Usado em
Anomalia **ativa**	Situação em `DETECTADA`, `EM_ANALISE`, `CONFIRMADA` ou `EM_CORRECAO`	§14, §15, §16
Anomalia **crítica**	Ativa **e** `risco = CRITICO`	§14, §15, §16
`podeReceberMissao(proposito)`	Anomalia ativa, **ou** `IRREVERSIVEL` com `proposito = INVESTIGACAO`	M1, M5, M6, M8

`ESTABILIZADA` e `IRREVERSIVEL` são finais (D26). Se a divergência reaparecer, registra-se uma **nova** anomalia; a antiga continua disponível para consulta (§5).

---

6. Operações compostas

Cada operação abaixo é uma única transação (§9, §20): se qualquer passo falhar, nada é gravado. Elas são os casos de uso de serviço do doc 01, seção 6.2. Toda mudança de situação de agente dentro delas grava também um `RegistroAgente` (seção 4.1). Os registros do histórico da anomalia ligados a uma missão levam o `missao_id` (doc 02, D37).

6.1 Iniciar missão (M5/M6)

1. Conferir as guardas: `podeIniciar()` e, a partir de `PLANEJADA`, `¬exigeAutorizacao()`.
2. Missão → `EM_EXECUCAO`; `dataInicio = agora`.
3. Cada agente designado → `EM_MISSAO` (G2). O `@Version` do agente detecta início concorrente em outra missão (D11).
4. Anomalia → `EM_ANALISE` (N2) ou `EM_CORRECAO` (N3), conforme o `proposito`. Se ela for `IRREVERSIVEL`, não muda (D26).
5. Registros `MISSAO_INICIADA` e, se a anomalia mudou, `MUDANCA_SITUACAO`.

6.2 Concluir ou registrar falha (M10/M11)

1. Conferir a situação final contra a tabela 5.2.
2. Criar o `Encerramento`; missão → `CONCLUIDA` ou `FALHOU`; `dataFim = agora`.
3. Para cada participação: `dataSaida = agora`. Agente → `SUSPENSO` se a missão falhou e ele registrou intervenção `SEVERO` nela (G4); senão → `DISPONIVEL` (G3).
4. Anomalia → situação final (N4/N5).
5. Se a situação final for `ESTABILIZADA` ou `IRREVERSIVEL`, cancelar as missões pendentes que não são mais admissíveis (M8).
6. Registros `ENCERRAMENTO`, `MUDANCA_SITUACAO` e um `MISSAO_CANCELADA` por missão cancelada no passo 5.

6.3 Cancelar missão (M7/M8)

1. Missão → `CANCELADA`; `dataFim = agora`.
2. Participações: `dataSaida = agora`. Os agentes não mudam, pois continuam `DISPONIVEL`.
3. Registro `MISSAO_CANCELADA`, com o motivo.

6.4 Declarar anomalia irreversível (N6)

1. Conferir que não há missão `EM_EXECUCAO` para a anomalia.
2. Anomalia → `IRREVERSIVEL`.
3. Cancelar as missões de `CORRECAO` pendentes (M8). As de investigação continuam.

4. Registros `MUDANCA_SITUACAO`, com a justificativa na descrição (doc 01, D33), e um `MISSAO_CANCELADA` por missão cancelada.

6.5 Inativar agente (G7)

1. Agente → `INATIVO`, com registro de agente e o motivo informado.
2. Remover as designações do agente em missões pendentes (M3), já que ele não volta. Cada remoção gera `AGENTE_REMOVIDO` no histórico da anomalia daquela missão.
3. Desativar o usuário vinculado ao agente, se houver: um agente desligado não acessa mais o sistema (doc 06).

6.6 Registrar ocorrência (M9)

A ocorrência fica no MongoDB e o histórico no PostgreSQL, que não compartilham transação. Para que nenhum dos dois se perca (§21, A14), a gravação usa o padrão *outbox* (doc 04, D40):

1. Conferir que a missão está `EM_EXECUCAO` e que o agente do usuário está designado nela (§11, doc 06).
2. Gerar na aplicação o identificador da ocorrência (`ObjectId`).
3. Na mesma transação do PostgreSQL: gravar o registro `OCORRENCIA` no histórico (com `missao_id` e o identificador em `valor_novo`) e a ocorrência completa no `outbox`.
4. Depois do *commit*, o processo de entrega grava a ocorrência no MongoDB pelo identificador, de forma idempotente, e marca o `outbox` como processado. Se o MongoDB estiver fora, a entrega é repetida mais tarde.

7. Invariantes

Estas condições valem em **qualquer** estado do sistema que as transições acima conseguem produzir. Elas formalizam o "estados incompatíveis" do §9.

#	Invariante	Garantida em
I1	No máximo uma missão `EM_EXECUCAO` por anomalia	Banco: `uk_missao_em_execucao` (D23)
I2	Anomalia `EM_ANALISE` ⇔ há uma investigação em execução nela; `EM_CORRECAO` ⇔ há uma correção em execução (exceto investigação em `IRREVERSIVEL`)	Operações 6.1 e 6.2
I3	Nenhuma correção em execução sobre anomalia `IRREVERSIVEL`	Guarda de M5/M6 + I7
I4	Agente `EM_MISSAO` ⇔ participa de exatamente uma missão `EM_EXECUCAO`	Operações 6.1 e 6.2 + `@Version`
I5	Missão `EM_EXECUCAO` tem ≥ 1 agente	Guarda de M5/M6; M3 só antes do início
I6	Missão encerrada não tem participação aberta (`dataSaida` nula)	Operações 6.2 e 6.3
I7	Missão pendente sempre admissível pela sua anomalia	Guarda de M1 + M8 (D24)
I8	Agente `INATIVO` não fica designado em missão pendente	Operação 6.5

E estas valem em **toda transição**:

#	Propriedade
P1	Anomalia em estado final nunca muda de situação
P2	Falha nunca leva a anomalia a `ESTABILIZADA` (§13)
P3	Missão que exige autorização nunca vai de `PLANEJADA` para `EM_EXECUCAO` (§8)

#	Propriedade
P4	Missão encerrada nunca muda
P5	Agente `INATIVO` nunca muda

8. Verificação

As regras foram verificadas com um modelo executável (`verificacao/modelo-estados.py`). Ele percorre **todos** os estados alcançáveis de um mundo com 1 anomalia, 2 missões e 2 agentes, combinando propósitos, prioridades e riscos, e checa I1–I8 e P1–P5 em cada transição.

```
python3 docs/verificacao/modelo-estados.py      # regras deste documento
python3 docs/verificacao/modelo-estados.py v1   # rascunho inicial, com as falhas encontradas
```

8.1 O que a revisão encontrou

O rascunho v1 (sem D23, D24 e D25) explorou 250.144 estados e violou cinco invariantes:

Achado	Caminho mínimo	Correção
Duas missões em execução na mesma anomalia; ao fim de uma, a anomalia ficava `ESTABILIZADA` com a outra ainda rodando, ou saía de `ESTABILIZADA` (I1, I2, I3, P1)	criar m0 → designar → iniciar → criar m1 → designar → iniciar	D23
Correção `PLANEJADA` sobre anomalia `IRREVERSIVEL` ou `ESTABILIZADA` (I7)	criar m1 (correção) → declararIrreversivel	D24
Missão sem saída além de cancelar, quando um agente designado é suspenso e outro está disponível	criar → designar a0, a1 → autorizar → suspender a0	D25

O v2, com as regras deste documento, explorou 135.536 estados **sem nenhuma violação**, e nenhuma missão pendente fica travada enquanto houver agente disponível.

8.2 Uso nos testes (§23)

Cada linha das tabelas 3.1, 4.1 e 5.1 vira pelo menos dois testes de entidade: a transição permitida e a guarda violada (`409`). Os caminhos da tabela 8.1 viram testes de integração das operações compostas, pois são exatamente os cenários que o rascunho errava.

8.3 Duas representações, uma verdade

Os diagramas existem em dois formatos: o Mermaid, que o GitHub renderiza sozinho, e o draw.io, mais fácil de editar e de apresentar. No draw.io, a Anomalia usa o superestado *Ativa* (A1), com `N7 alterarRisco` no compartimento, e as transições internas ficam dentro do próprio estado, como manda a UML.

Para que os dois não divirjam, rode antes de cada commit que mexer no `.drawio` ou neste documento:

```
python3 docs/verificacao/confere-diagramas.py
```

O script compara origem, destino e IDs de cada transição nas duas fontes, as transições internas e os IDs das tabelas 3.1, 4.1 e 5.1. Ele termina com código 1 se encontrar qualquer diferença.

9. Decisões de modelagem

D23 — Uma missão em execução por anomalia

A situação da anomalia descreve **o** atendimento em curso. Com duas missões simultâneas, o encerramento da primeira mudaria a situação da anomalia por baixo da segunda (achado 1 da seção 8.1). Missões adicionais podem ser **planejadas**, mas só iniciam quando a anterior encerra.

- No banco: `uk_missao_em_execucao`, um índice único parcial em `missao (anomalia_id) WHERE situacao = 'EM_EXECUCAO'` (doc 02). Também protege contra dois inícios concorrentes.
- *Descartada*: permitir várias e derivar a situação da anomalia das missões ativas, o que torna a situação um valor calculado e complica o histórico do §12.

D24 — Missões pendentes inadmissíveis são canceladas automaticamente

Quando a anomalia chega a `ESTABILIZADA` ou `IRREVERSIVEL`, as missões pendentes que ela não admite mais são canceladas **na mesma transação** (M8). Deixá-las pendentes seria um estado incompatível (§9).

- *Descartada*: manter as missões e só bloquear o início, o que deixaria o operador com missões impossíveis na lista.

D25 — Remover designação apaga a linha de `missao_agente`

Antes do início não há intervenção nem ocorrência ligada à participação, então apagar a linha não perde informação: a designação e a remoção ficam no histórico (`AGENTE_DESIGNADO`, `AGENTE_REMOVIDO`). Assim, o mesmo agente pode ser designado de novo sem violar `uk_missao_agente`.

- Depois do início, a participação nunca é removida, só encerrada (`dataSaida`).
- *Descartada*: marcar `dataSaida` antes do início, o que exigiria distinguir "saiu antes" de "saiu ao fim" e impediria a nova designação.

D26 — `ESTABILIZADA` e `IRREVERSIVEL` são finais

Uma anomalia resolvida ou perdida não volta a ser trabalhada. Se a divergência reaparecer, é uma nova anomalia, com o seu próprio histórico. A única missão que uma anomalia final aceita é a investigação em `IRREVERSIVEL`, que não muda a situação.

D27 — `EM_ANALISE` e `EM_CORRECAO` só por missão

Esses estados significam "há uma missão trabalhando nisto agora". Permitir colocá-los manualmente quebraria I2. As alterações manuais se limitam a `declararIrreversivel` (N6) e ao risco (N7).

D28 — A autorização vale para a missão, avaliada ao iniciar

Os dados da missão só mudam em `PLANEJADA`; a equipe pode mudar depois da autorização, com registro no histórico. `exigeAutorizacao()` é avaliada no início, porque o risco da anomalia pode mudar depois do planejamento.

- *Descartada* (A13, *confirmado em 2026-11-08*): voltar a missão para `PLANEJADA` a cada mudança de equipe.

D29 — `INATIVO` é final; `reativar()` só a partir de `SUSPENSO`

Suspensão é temporária; inativação é desligamento. O doc 01 não tinha uma forma de sair de `SUSPENSO`, então `Agente.reativar()` foi acrescentado.

D30 — Suspensão automática só por falha com intervenção severa

Fecha A4: o agente não volta a `DISPONIVEL` quando a missão **falhou** e **ele próprio** registrou nela uma intervenção com impacto `SEVERO`. Os outros agentes da mesma missão são liberados normalmente.

10. Pontos em aberto

Fechados neste documento

#	Questão	Decisão
A1	O que é anomalia ativa	Seção 5.3
A2	Transições de `suspender()` e `inativar()`	G5, G6, G7 e seção 4.2
A3	Regras de `cancelar()`	M7 e M8; só a partir de `PLANEJADA` / `AUTORIZADA`
A4	Exceção do §7 para o agente não voltar a `DISPONIVEL`	G4 e D30
A8	Transição da anomalia ao encerrar a missão	Tabela 5.2
A12	Onde ficam as mudanças de situação do agente	`RegistroAgente` (seção 4.1; doc 01, D31)
A13	Mudar a equipe depois da autorização exige nova autorização?	Não (D28, confirmado)
A14	Como gravar a ocorrência e o histórico sem perder nenhum	*Outbox* (operação 6.6; doc 04, D40)

Não há pontos em aberto neste documento. A5 foi fechado no doc 01.

11. Histórico de revisões

Versão	Data	Alteração
1.0	2026-10-21	Versão inicial: máquinas de Missão, Agente e Anomalia; operações compostas; invariantes verificados por modelo executável; D23–D30; fecha A1, A2, A3, A4 e A8
1.1	2026-10-21	Diagramas também no draw.io (páginas *Estados — **), com superestado *Ativa* e transições internas em compartimento; conferência automática draw.io × Mermaid × tabelas
1.2	2026-11-08	Ajustes vindos da implementação: registro de agente em toda transição G (fecha A12), `suspenderPorFalha()` em G4, motivo em G6, parâmetros de N1, operação 6.6 com *outbox* (fecha A14), passo 3 da operação 6.5; A13 confirmado